

SQLite:

```
pip install flask flask_sqlalchemy
# SQLite database configuration
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///flask_crud.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

MySQL:

```
pip install flask flask_sqlalchemy pymysql
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://root:@localhost/flask_crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Postgres:

```
pip install flask flask_sqlalchemy psycopg2-binary
# PostgreSQL database configuration
app.config['SQLALCHEMY_DATABASE_URI'] =
'postgresql+psycopg2://postgres:password@localhost:5432/flask_crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

SQLAlchemy (an ORM tool) for mysql database

- Using SQLAlchemy ORM in Flask is cleaner, more secure, and scalable.
- Here's how to perform CRUD operations in Flask using MySQL with SQLAlchemy ORM and HTML templates.

1. Install Required Packages:

```
pip install flask flask_sqlalchemy pymysql
```

2. Create MySQL Database:

```
CREATE DATABASE flask_crud;
```

3. Project Structure

```
/flask_crud_app
|
├── app.py
├── templates/
│   ├── index.html
│   ├── create.html
│   └── update.html
```

app.py — Flask + SQLAlchemy ORM

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
```

```
app = Flask(__name__)
```

```

# MySQL connection (using PyMySQL driver)
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://root:@localhost/flask_crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# Model
class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f"<Student {self.name}>"

# READ
@app.route('/')
def index():
    students = Student.query.all()
    return render_template('index.html', students=students)

# CREATE
@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        new_student = Student(name=name, email=email)
        db.session.add(new_student)
        db.session.commit()
        return redirect(url_for('index'))
    return render_template('create.html')

# UPDATE
@app.route('/update/<int:id>', methods=['GET', 'POST'])
def update(id):
    student = Student.query.get_or_404(id)
    if request.method == 'POST':
        student.name = request.form['name']
        student.email = request.form['email']
        db.session.commit()
        return redirect(url_for('index'))
    return render_template('update.html', student=student)

# DELETE
@app.route('/delete/<int:id>')
def delete(id):
    student = Student.query.get_or_404(id)

```

```

    db.session.delete(student)
    db.session.commit()
    return redirect(url_for('index'))

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)

```

HTML Templates

templates/index.html

```

<!doctype html>
<html>
<head><title>Student List</title></head>
<body>
    <h2>Student List</h2>
    <a href="{{ url_for('create') }}">Add Student</a>
    <table border="1">
        <tr><th>ID</th><th>Name</th><th>Email</th><th>Actions</th></tr>
        {% for student in students %}
        <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.email }}</td>
            <td>
                <a href="{{ url_for('update', id=student.id) }}">Edit</a> |
                <a href="{{ url_for('delete', id=student.id) }}">Delete</a>
            </td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>

```

templates/create.html

```

<!doctype html>
<html>
<head><title>Add Student</title></head>
<body>
    <h2>Add Student</h2>
    <form method="POST">
        Name: <input type="text" name="name" required><br>
        Email: <input type="email" name="email" required><br>
        <input type="submit" value="Add">
    </form>

```

```

    </form>
    <a href="{{ url_for('index') }}">Back</a>
</body>
</html>

```

templates/update.html

```

<!doctype html>
<html>
<head><title>Edit Student</title></head>
<body>
    <h2>Edit Student</h2>
    <form method="POST">
        Name: <input type="text" name="name" value="{{ student.name }}" required><br />
        Email: <input type="email" name="email" value="{{ student.email }}" required><br />
        <input type="submit" value="Update">
    </form>
    <a href="{{ url_for('index') }}">Back</a>
</body>
</html>

```

Relationships

```

class MenuItem(db.Model):
    __tablename__ = 'MenuItem'

    id = Column(Integer, primary_key=True)
    name = Column(String(100))
    price = Column(DECIMAL(10, 2))
    availability = Column(Boolean)
    category_id = Column(Integer, ForeignKey('Category.id'))

    category = relationship("Category", back_populates="menu_items")

class Category(db.Model):
    __tablename__ = 'Category'

    id = Column(Integer, primary_key=True)
    category_name = Column(String(50))
    menu_items = relationship("MenuItem", back_populates="category")

class Users(db.Model):
    __tablename__ = 'Users'

    id = Column(Integer, primary_key=True)

```

```
name = Column(String(100))
email = Column(String(100))
password = Column(String(256))
role = Column(String(20))
```

```
orders = relationship("Orders", back_populates="user")
```

```
class Orders(db.Model):
```

```
    __tablename__ = 'Orders'
```

```
    id = Column(Integer, primary_key=True)
```

```
    user_id = Column(Integer, ForeignKey('Users.id'))
```

```
    total_amount = Column(DECIMAL(10, 2))
```

```
    order_date = Column(DateTime)
```

```
    order_status = Column(String(20))
```

```
    user = relationship("Users", back_populates="orders")
```

```
    order_items = relationship("OrderItem", back_populates="order")
```

```
    payments = relationship("Payment", back_populates="order")
```

```
class OrderItem(db.Model):
```

```
    __tablename__ = 'OrderItem'
```

```
    id = Column(Integer, primary_key=True)
```

```
    order_id = Column(Integer, ForeignKey('Orders.id'))
```

```
    item_id = Column(Integer)
```

```
    quantity = Column(Integer)
```

```
    rate = Column(Integer)
```

```
    order = relationship("Orders", back_populates="order_items")
```

```
class Payment(db.Model):
```

```
    __tablename__ = 'Payment'
```

```
    id = Column(Integer, primary_key=True)
```

```
    order_id = Column(Integer, ForeignKey('Orders.id'))
```

```
    amount = Column(DECIMAL(10, 2))
```

```
    payment_method = Column(String(20))
```

```
    payment_status = Column(String(20))
```

```
    paid_amount = Column(DECIMAL(10, 2))
```

```
    discount = Column(DECIMAL(10, 2))
```

```
    total_amount = Column(DECIMAL(10, 2))
```

```
    order = relationship("Orders", back_populates="payments")
```